

RAPPORT DE SÉCURITÉ - TP FINAL

Étudiant : [Votre nom]

Date : [Date du TP]

Durée : 4 heures

TABLE DES MATIÈRES

- 1. Synthèse des vulnérabilités
- 2. Corrections détaillées
- 3. Tests de validation
- 4. Conclusion

1. SYNTHÈSE DES VULNÉRABILITÉS

Tableau récapitulatif

#	Fonction	Ligne(s)	Type	Gravité	Corrigée
1					☐ Oui ☐ Non
2					☐ Oui ☐ Non
3					☐ Oui ☐ Non
4					☐ Oui ☐ Non
5					☐ Oui ☐ Non
6					☐ Oui ☐ Non
7					☐ Oui ☐ Non
8					☐ Oui ☐ Non
9					☐ Oui ☐ Non
10					☐ Oui ☐ Non

Rajouter des lignes si nécessaire.

Total de vulnérabilités identifiées : _____

2. CORRECTIONS DÉTAILLÉES

Chaque correction doit contenir les 4 sections obligatoires.

Correction #1 : [Titre de la vulnérabilité]

Vulnérabilité originale

Fonction concernée : `nom_fonction()`

Ligne(s) : XX-YY

Description :

[Décrivez précisément le problème de sécurité]

Code vulnérable :

```
// Collez le code problématique ici
```

Attaque possible

Scénario d'exploitation :

[Décrivez comment un attaquant pourrait exploiter cette vulnérabilité]
[Donnez un exemple concret d'entrée malicieuse]

Impact :

- ☐ Exécution de code arbitraire
- ☐ Lecture de mémoire
- ☐ Dénî de service (crash)
- ☐ Élévation de privilèges
- ☐ Fuite d'informations
- ☐ Autre : _____

Correction implémentée

Code corrigé :

```
// Collez votre code sécurisé ici
```

Modifications effectuées :

1. [Listez les changements]
- 2.
- 3.

Justification de sécurité

Pourquoi cette correction est sûre :

[Expliquez pourquoi votre solution empêche l'exploitation]
[Mentionnez les mécanismes de sécurité utilisés]

Tests effectués :

Commandes de test utilisées

Correction #2 : [Titre de la vulnérabilité]**● Vulnérabilité originale****Fonction concernée :****Ligne(s) :****Description :****Code vulnérable :****🔥 Attaque possible****Scénario d'exploitation :****Impact :**

- ☐

✅ Correction implémentée**Code corrigé :****Modifications effectuées : 1.****🛡 Justification de sécurité****Pourquoi cette correction est sûre :****Tests effectués :**

Correction #3 : [Titre de la vulnérabilité]

 Vulnérabilité originale

Fonction concernée :

Ligne(s) :

Description :

Code vulnérable :

 Attaque possible

Scénario d'exploitation :

Impact :

- ☐

 Correction implémentée

Code corrigé :

Modifications effectuées : 1.

 Justification de sécurité

Pourquoi cette correction est sûre :

Tests effectués :

Correction #4 : [Titre de la vulnérabilité]

[Répétez le format ci-dessus]

Correction #5 : [Titre de la vulnérabilité]

[Répétez le format ci-dessus]

Correction #6 : [Titre de la vulnérabilité]

[Répétez le format ci-dessus]

Correction #7 : [Titre de la vulnérabilité]

[Répétez le format ci-dessus]

Correction #8 : [Titre de la vulnérabilité]

[Répétez le format ci-dessus]

Correction #9 : [Titre de la vulnérabilité]

[Répétez le format ci-dessus]

Correction #10 : [Titre de la vulnérabilité]

[Répétez le format ci-dessus]

3. TESTS DE VALIDATION

3.1 Compilation

Commande utilisée :

```
gcc -o secure secure.c -lcrypto -Wall -Wextra
```

Résultat :

```
[Collez la sortie de compilation]
```

Nombre de warnings : _____

Nombre d'erreurs : _____

3.2 Tests fonctionnels

Fonctionnalité	Test effectué	Résultat
Création utilisateur	Tentative avec nom très long	☐ OK ☐ KO
Connexion	Login avec mauvais credentials	☐ OK ☐ KO
Transfert	Montant négatif	☐ OK ☐ KO
Backup	Nom de fichier avec <code>../</code>	☐ OK ☐ KO
Commande système	Injection avec <code>;</code>	☐ OK ☐ KO

3.3 Analyse mémoire (Valgrind)

Commande :

```
valgrind --leak-check=full --show-leak-kinds=all ./secure
```

Résultat :

[Collez la sortie de valgrind]

Fuites détectées : _____ bytes

Blocs non libérés : _____

3.4 Tests de sécurité

Test 1 - Buffer Overflow

```
# Entrée testée :  
# Résultat attendu :  
# Résultat obtenu :
```

Test 2 - Format String

```
# Entrée testée :  
# Résultat attendu :  
# Résultat obtenu :
```

Test 3 - Command Injection

```
# Entrée testée :  
# Résultat attendu :
```

Résultat obtenu :

4. CONCLUSION

4.1 Résumé des corrections

Statistiques :

- Vulnérabilités CRITIQUES corrigées : _____
- Vulnérabilités HAUTES corrigées : _____
- Vulnérabilités MOYENNES corrigées : _____
- Total de lignes modifiées : ~ _____

Principaux changements : 1. 2. 3.

4.2 Difficultés rencontrées

Problème 1 :

[Description du problème]
[Comment vous l'avez résolu]

Problème 2 :

[Description du problème]
[Comment vous l'avez résolu]

4.3 Améliorations possibles

Sécurité :

- ☐ Implémenter un système de salage pour les mots de passe
- ☐ Ajouter des logs de sécurité
- ☐ Limiter le nombre de tentatives de connexion
- ☐ Implémenter un système de sessions avec timeout
- ☐ Autre : _____

Fonctionnalité :

- ☐ Chiffrement des données sensibles
 - ☐ Audit trail complet
 - ☐ Protection contre les timing attacks
 - ☐ Autre : _____
-

4.4 Apprentissages clés

Ce que j'ai appris : 1. 2. 3.

Compétences développées :

- ☐ Identification de vulnérabilités
- ☐ Utilisation de fonctions sécurisées
- ☐ Validation d'entrées
- ☐ Gestion mémoire rigoureuse
- ☐ Cryptographie appliquée

4.5 Auto-évaluation

Critère	Note estimée /20	Justification
Analyse	_____/6	
Corrections	_____/10	
Documentation	_____/4	
TOTAL	_____/20	

ANNEXES

Annexe A : Checklist de vérification finale

- ☐ Tous les `gets()` remplacés par `fgets()`
- ☐ Tous les `scanf("%s")` sécurisés avec taille maximale. Exemple : `scanf("%19s", buffer)` pour un buffer de 20 bytes
- ☐ Tous les `strcpy()` remplacés par `strncpy()`
- ☐ Tous les `strcat()` remplacés par `strncat()`
- ☐ Tous les `sprintf()` remplacés par `snprintf()`
- ☐ Cryptographie utilise SHA-256
- ☐ Pas de mots de passe en clair
- ☐ Pas de backdoors
- ☐ Tous les indices de tableaux validés
- ☐ Tous les `malloc()` ont un `free()` correspondant
- ☐ 0 fuites mémoire (valgrind)
- ☐ 0 warnings de compilation
- ☐ Validation stricte des noms de fichiers
- ☐ Protection contre command injection

Annexe B : Références utilisées

- 1.
- 2.
- 3.

FIN DU RAPPORT